

3.7 OneToOne, OneToMany, ManyToOne, ManyToMany Mappings in Spring Data JPA

- **One-to-One Mapping in Spring Data JPA:**

- A **One-to-One** relationship means:
 - One record in Table A is associated with exactly one record in Table B, and vice versa.
- **Example:**
 - One **User** ↔ One **Profile**
 - One **Employee** ↔ One **Address**
 - One **Customer** ↔ One **Wallet**
- **Unidirectional One-to-One Mapping:**

```
@Entity
public class Employee {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToOne
    @JoinColumn(name = "address_id") // FK column
    private Address address;
}
```

```
@Entity
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String city;
    private String state;
}
```

- **@OneToOne** → defines one-to-one relationship
- **@JoinColumn** → creates a **foreign key** in employee table
- `employee.address_id` → references `address.id`

- **Bidirectional One-to-One Mapping:**

```
@Entity
public class Employee {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToOne
    @JoinColumn(name = "address_id")
    private Address address;
}
```

```
@Entity
public class Address {
    @Id
    @GeneratedValue
    private Long id;
    private String city;
    private String state;
    @OneToOne(mappedBy = "address")
    private Employee employee;
}
```

- `mappedBy = "address"` → field name in **Employee**.
- **No additional foreign key created.**
- Only **one FK exists** (in employee table).

3.7 OneToOne, OneToMany, ManyToOne, ManyToMany Mappings in Spring Data JPA

- Owning Side vs Inverse Side:

- Owning Side:

- The entity that **contains** the **foreign key**.
 - **Controls** the relationship.
 - Uses **@JoinColumn**.

- Inverse Side:

- Does **not contain** foreign key.
 - Uses **mappedBy**.

- One-to-Many & Many-to-One Mapping in Spring Data JPA:

- Many-to-One:

- Many records in one table are associated with **one record** in another table.

- Example:

- Many **Employees** belong to one **Department**
 - Many **Orders** belong to one **Customer**

- One-to-Many:

- One record in a table is associated with **many records** in another table.

- Example:

- One **Department** has many **Employees**
 - One **Customer** has many **Orders**

- ⚠ These two mappings usually represent **two sides** of the **same relationship**.

- Database Perspective:

- Relational databases do **not** have One-to-Many directly.
 - They always use **a foreign key** on the **MANY side**.
 - So in **JPA**:
 - **@ManyToOne** is the **owning side**.
 - **@OneToMany** is the **inverse** (mapped) **side**.

- Many-to-One Mapping (Owning Side):

```
@Entity
public class Employee {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

3.7 OneToOne, OneToMany, ManyToOne, ManyToMany Mappings in Spring Data JPA

- **Explanation:**

- `@ManyToOne` → **many** employees belong **to one** department
- `@JoinColumn` → foreign key column in employee table
- This side **owns** the relationship

- **One-to-Many Mapping (Inverse Side):**

```
@Entity
public class Department {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    private List<Employee> employees;
}
```

- **Explanation:**

- mappedBy = "department" → refers to **field** in Employee.
- **No foreign key** is created here.
- This side does **NOT own** the relationship.

- **Bidirectional One-to-Many / Many-to-One:**

- **Complete Mapping:**

```
// Employee (Owning Side)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department;
```

```
// Department (Inverse Side)
@OneToMany(mappedBy = "department")
private List<Employee> employees;
```

- ✓ One foreign key
- ✓ Clean bidirectional navigation

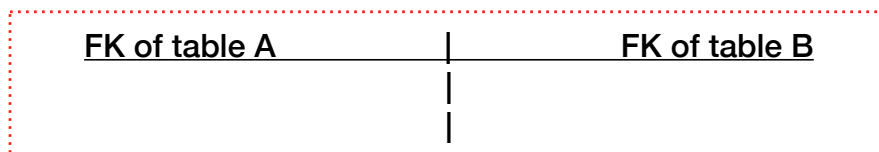
- **Many-to-Many Mapping in Spring Data JPA:**

- A **Many-to-Many** relationship means: **Multiple records** in Table **A** are associated with **multiple records** in Table **B**, and vice versa.
- Real-world **examples**:
 - Students ↔ Courses
 - Users ↔ Roles
 - Products ↔ Categories

3.7 OneToOne, OneToMany, ManyToOne, ManyToMany Mappings in Spring Data JPA

- **Database Perspective:**

- Relational databases do **not** support **Many-to-Many** directly. They use a **junction** (join) table.
- Join Table contains:
 - FK to Table A
 - FK to Table B



- **Bidirectional Many-to-Many:**

```
@Entity
public class Student { // Owing side

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses;
}
```

```
@Entity
public class Course { // Inverse side

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students;
}
```

- **@JoinTable** → defines the **junction table**.
- **joinColumns** → FK of **owning** entity.
- **inverseJoinColumns** → FK of **other** entity.
- Only **one** side **owns** the relationship.
- The owning side defines **@JoinTable**.
- Inverse side uses **mappedBy**.